

8 Divide and Conquer

In the last chapter we saw the power of reductions for proving problems undecidable. In this chapter and the next we will see the power of reduction for creating efficient algorithms for computable functions. Divide and conquer is a technique closely related to recursion where we solve a problem by reducing it to a smaller version of the exact same problem.

In Sections 8.1 through Section 8.3 we'll look at some common algorithms which use divide and conquer. In Section 8.4 we will discuss *recurrences*, which allow us to express the runtime of divide and conquer algorithms, and in Section 8.5, we'll see how to convert them into the normal asymptotic bounds we're used to. In the next chapter we will explore another important application of the divide and conquer technique.

8.1 Binary Search

Binary search is one of the simplest and most versatile algorithms. It's an algorithm you may have used yourself before even learning to program. For example, suppose that your professor tells you to turn to page 394 in your textbook. In order to find the appropriate page, you start by opening to the middle of the textbook, and see that you're on page 250. This comes before 394, so you keep a finger in page 250 and flip to somewhere that's about in the middle of the second half of the book, which gets you to page 375. This still isn't far enough, so you hold page 375 and flip to halfway through the remainder of the book, getting page 437. Now you've gone too far, so you find somewhere halfway between 375 and 437, page 406. After also visiting pages 390 and 398, you finally arrive at page 394, having had to look at 7 total page numbers on the way. This is much more efficient than if you had simply flipped through the pages one by one, looking at all of the first 394 pages to find the one you wanted.

The idea of binary search is to rapidly find some desired object in a large search space by repeatedly performing checks that allow you to divide the search space in half (i.e. halve the number of possible solutions you need to consider). The canonical application of binary search is finding an entry in a sorted array. This standard binary search algorithm is given below:

```
BINARYSEARCH( $A[1, \dots, n], x$ )
 $\ell \leftarrow 1$ 
 $u \leftarrow n$ 
While  $\ell \leq u$ :
     $m \leftarrow \lfloor (\ell + u)/2 \rfloor$ 
    If  $A[m] = x$ :
        Return True
    If  $A[m] > x$ :
         $u \leftarrow m - 1$ 
    Else:
         $\ell \leftarrow m + 1$ 
Return False
```

Theorem 41. *For every sorted array of integers A and every integer x , BINARYSEARCH returns True if and only if x is in the array A . Furthermore the runtime of BINARYSEARCH is $O(\log n)$.*

Proof. We begin by proving the correctness. We will prove the following loop invariant for the while loop: At the start of each iteration of the loop, if x is in A then x is in $A[\ell : u]$ (i.e. we will induct on the number of iterations of the while loop).

As a base case, note that after 0 iterations of the while loop (i.e. before the first iteration), $\ell = 1$ and $u = n$, so if x is in A then it is in $A[\ell : u]$.

Now suppose as an induction hypothesis that at the start of some iteration of the loop the invariant holds. We consider three cases:

- Suppose $A[m] = x$: Then $x \in A$ and the algorithm correctly returns True and there is no next iteration of the loop.

- Suppose $A[m] > x$: Then, since the array is sorted, every element in $A[m : u]$ is greater than x . Thus, if x was in $A[\ell : u]$ then x must be in $A[\ell : m - 1]$, so if the loop invariant held at the start of this loop iteration it also holds at the start of the next iteration.
- Suppose $A[m] < x$: Then, since the array is sorted, every element in $A[\ell : m]$ is less than x . Thus, if x was in $A[\ell : u]$ then x must be in $A[m + 1 : u]$, so if the loop invariant held at the start of this loop iteration it also holds at the start of the next iteration.

Thus, our invariant holds at the start of every iteration. We note that $u - \ell$ begins at $n - 1$ and decreases with every iteration of the loop where we don't return True (because either ℓ increases or u decreases). Therefore, after a finite number of iterations either we have returned True already or we have $u - \ell < 0$, meaning that $u < \ell$. At this point, our loop invariant implies that x is not in A and the algorithm correctly returns False. Therefore the algorithm is correct because it always terminates in finite time (never enters an infinite loop) and whenever it returns a value that value is correct.

Now we analyze the runtime. Since u and ℓ are always between 1 and n , all of the arithmetic in the loop is with positive integers less than or equal to n (i.e. numbers whose value is polynomial in the input length) so we can perform the arithmetic in the loop in constant time, and thus each loop iteration runs in $O(1)$ time.

We note that the size of our search interval, $A[\ell : u]$ is initially n . In each iteration of the loop where we do not return, the size of this interval decreases by a factor of at least 2. By definition, $\log_2(n)$ is the number of times that we need to divide n by 2 in order to get down to 1, so after $O(\log n)$ iterations of the loop, if we have not already returned, the size of $A[\ell : u]$ will be 0. At this point, we exit the loop and return False. Since each iteration is constant time, the total runtime for the algorithm is $O(\log n)$.

We note that the theorem only asks us to prove a big-O bound so we do not need to give an example where the algorithm takes $\Omega(\log n)$ time. $\square$

It's important to note that the correctness argument for binary search crucially hinges on the fact that the input array is *sorted*. Without this assumption, we can't do anything better than a $\Theta(n)$ time linear search (i.e. just iterate through every element of the array and check if it's the one we're looking for).

8.2 Applications of Binary Search

Binary search has applications far beyond just searching in arrays. In this section we will look at a few of them.

In Section 3.2 we saw the following algorithm for computing $\lfloor \sqrt{n} \rfloor$ for any positive integer n :

```

INTEGERSQUAREROOT( $x$ ):
   $i \leftarrow 0$ 
  While  $(i + 1)^2 \leq x$ :
     $i \leftarrow i + 1$ 
  Return  $i$ 

```

This algorithm works, but it's not particularly efficient. The while loop runs for $\Theta(\sqrt{x})$ iterations. Since the length of the input is $n \in \Theta(\log x)$, this means the number of iterations is $\sqrt{2^n} = 2^{n/2}$, so the algorithm is exponential time before even considering the time for computing $(i + 1)^2$.

We can use the idea behind binary search to create a much more efficient algorithm for this problem, which avoids squaring every positive integer less than $\sqrt{x}$.

```

FASTSQUAREROOT( $x, \ell \leftarrow 0, u \leftarrow x$ ):
    If  $\ell = u$ :
        Return  $\ell$ 
     $m \leftarrow \lfloor (\ell + u)/2 \rfloor$ 
    If  $(m + 1)^2 \leq x$ :
        Return FASTSQUAREROOT( $x, m + 1, u$ )
    Else:
        Return FASTSQUAREROOT( $x, \ell, m$ )

```

Theorem 42. For every positive integer x , FASTSQUAREROOT(x) returns $\lfloor \sqrt{x} \rfloor$. Furthermore, FASTSQUAREROOT runs in time $O(n^3)$.

Proof. We first prove by induction on $u - \ell$ that if $\ell \leq \lfloor \sqrt{x} \rfloor \leq u$ then FASTSQUAREROOT(x, ℓ, u) returns $\lfloor \sqrt{x} \rfloor$.

- Base Case: If $\ell = u$ and $\ell \leq \lfloor \sqrt{x} \rfloor \leq u$ then $\ell = \lfloor \sqrt{x} \rfloor$, so when the algorithm returns ℓ it returns $\lfloor \sqrt{x} \rfloor$.
- Induction Hypothesis: Suppose that for all $\ell', u' \in \mathbb{Z}_{\geq 0}$ with $u' - \ell' < u - \ell$ and $\ell' \leq \lfloor \sqrt{x} \rfloor \leq u'$ we have that FASTSQUAREROOT(x, ℓ', u') returns $\lfloor \sqrt{x} \rfloor$.
- Induction Step: Let $\ell \leq \lfloor \sqrt{x} \rfloor \leq u$. We note that $\ell \leq m < u$. We consider two cases:
 - If $(m + 1)^2 \leq x$, then $m + 1 \leq \sqrt{x}$, and $u - (m + 1) \leq u - (\ell + 1) < u - \ell$, so by our IH, FASTSQUAREROOT($x, m + 1, u$) returns $\lfloor \sqrt{x} \rfloor$.
 - Otherwise, $(m + 1)^2 > x$, so $\sqrt{x} < m + 1$ and thus $\lfloor \sqrt{x} \rfloor \leq m$. Furthermore, $u - m < u - \ell$, so by our IH FASTSQUAREROOT(x, ℓ, m) returns $\lfloor \sqrt{x} \rfloor$.

In either case, our recursive call returns $\lfloor \sqrt{x} \rfloor$, and therefore FASTSQUAREROOT(x, ℓ, u) does as well.

To finish proving the correctness of FASTSQUAREROOT we note that when we first call FASTSQUAREROOT(x) the default parameters of $\ell = 0$ and $u = x$ satisfy $\ell \leq \lfloor \sqrt{x} \rfloor \leq u$, so by our inductive proof, FASTSQUAREROOT(x) returns $\lfloor \sqrt{x} \rfloor$.

Now we analyze the runtime. As in analysis of binary search, we note that each recursive call $u - \ell$ decreases by a factor of 2, so since $u - \ell$ is initially x after $\Theta(\log x) = \Theta(n)$ recursive calls we will reach a call where $\ell = u$ and the function immediately returns. Before recursing each call to FASTSQUAREROOT has to compute $\lfloor (\ell + u)/2 \rfloor$ and $(m + 1)^2$. Since $\ell, u, m \leq x$, these numbers have length $O(n)$. Adding two n -digit numbers takes time $\Theta(n)$ and multiplying two n -digit numbers with the grade school algorithm (multiply each pair of digits) takes time $\Theta(n^2)$, so each call to FASTSQUAREROOT does $O(n^2)$ work outside of the recursive calls, and the total runtime of the algorithm is $O(n) \cdot O(n^2) = O(n^3)$. (We will discuss algorithms for arithmetic on large numbers in more detail in Chapter 9) $\square$

Sidenote: We could improve the runtime of this algorithm using one of the faster multiplication algorithms that we will discuss next chapter, but improving the runtime from $\Omega(2^{n/2})$ to $O(n^3)$ is already an exponential improvement over our previous square root algorithm.

The core property that allowed us to speed up the algorithm with the binary search technique is that squaring is a *monotone* function (over the non-negative integers), meaning that if $a > b$, then $a^2 > b^2$. Because of this, whenever we have the possible values for the square root narrowed down to a range we can pick a value m in the middle of the range and check how m^2 compares with x . If m^2 is bigger than x , then we can immediately rule out every number bigger than m , because their squares will also be bigger than x . Similarly, if m^2 is smaller than x , then we can rule out every number smaller than m , because their squares will be smaller than m^2 . Either way, we cut our search space in half. This same “cut the search space in half” technique can be applied to a wide variety of problems.

Sometimes, we don't have an upper bound on the size of the space that we're searching. In this case, we can use a variant of binary search known as exponential search. For example, suppose that we are trying to determine the maximum weight that a bridge can support, rounded to the nearest kilogram. To help us, we have a function `SIMULATEWEIGHT` which takes as input a number of kilograms, simulates putting that much weight on the bridge, and returns whether or not the bridge survived (this is obviously a very simplified model of civil engineering). We would like to figure out the maximum weight the bridge can support using as few calls as possible to `SIMULATEWEIGHT`. Assuming that we have no clue how much weight the bridge can support, what strategy should we use?

Unlike in the square root problem, here we aren't given an upper bound, so we can't start halfway in-between the upper and lower bound. Instead, we can use the following approach (assuming that our bridge can support at least 1kg):

```

FINDMAXWEIGHT()
   $i \leftarrow 1$ 
  While SIMULATEWEIGHT( $i$ )
     $i \leftarrow 2i$ 
  Binary search between  $i/2$  and  $i$  for the largest integer for which SIMULATEWEIGHT returns True

```

Here we repeatedly double i until we find a value which is larger than the maximum weight that the bridge can support. When we break out of the while loop we know that i kilograms break the bridge, but $i/2$ do not, so we can binary search in the range $i/2$ to i to find the maximum weight the bridge supports. If the maximum weight that the bridge can support is y , then we use $\Theta(\log y)$ calls to `SIMULATEWEIGHT` to find an $i > y$ and $\Theta(\log y)$ calls to `SIMULATEWEIGHT` to binary search for y in the range from $i/2$ to i , so in total our algorithm makes $\Theta(\log y)$ calls to `SIMULATEWEIGHT`. Note that for this problem we have to express our runtime in terms of the size of the answer rather than the size of the input (`FINDMAXWEIGHT` doesn't even use an input!). Also note that once again the reason we can use binary search is because our function `SIMULATEWEIGHT` is monotone. If i kilograms of weight break the bridge, then we know that j kilograms will as well for any $j > i$.

We simply wrote "binary search" here rather than writing out the full details of a binary search algorithm. Because binary search is such a broadly applicable algorithm, it is common to use it as a subroutine in other algorithms. The details of maintaining the upper and lower bounds, and the inductive proof of correctness are a bit tedious to go through every time we want to use this algorithm. In your algorithms you may feel free to call binary search as a subroutine, and assume its correctness. Be careful, however, to make sure that the input you're searching through actually meets the preconditions for binary search (i.e. you're searching a sorted array, or trying to find a threshold for a monotone function).

8.3 Mergesort

Binary search involves reducing a problem to a single smaller version of the same problem. Having only one subproblem to solve is efficient, but not always possible. Sometimes the best we can do is to reduce the original problem to two or more smaller versions. In this case, once we've recursively solved the smaller problems, we have multiple answers and we need to combine them in some way to obtain the answer to the original problem.

A simple divide and conquer algorithm based on this approach is mergesort, developed by John von Neumann in 1945. We start by introducing the helper subroutine `MERGE` and proving its correctness and runtime.

```

MERGE( $A[1, \dots, m], B[1, \dots, n]$ )
  Result  $\leftarrow$  an empty array of length  $m + n$ 
   $i, j, k \leftarrow 1$ 
  While  $k \leq m + n$ :
    If  $i = m + 1$ :
      Result[ $k$ ]  $\leftarrow B[j]$ 
       $j \leftarrow j + 1$ 
    Else If  $j = n + 1$ :
      Result[ $k$ ]  $\leftarrow A[i]$ 
       $i \leftarrow i + 1$ 
    Else If  $A[i] < B[j]$ :
      Result[ $k$ ]  $\leftarrow A[i]$ 
       $i \leftarrow i + 1$ 
    Else:
      Result[ $k$ ]  $\leftarrow B[j]$ 
       $j \leftarrow j + 1$ 
     $k \leftarrow k + 1$ 
  Return Result

```

Theorem 43. If A and B are sorted arrays of integers, then $\text{MERGE}(A, B)$ returns a sorted array containing all the elements of A and B combined. Furthermore, $\text{MERGE}(A, B)$ runs in time $O(m + n)$.

Proof. We note that k keeps track of the next position that we have to fill in the merged array Result, while i keeps track of the index of the first element of A that we have yet to include in the merged list and j keeps track of the first element of B that we have yet to include in the merged list. Since A and B are sorted, the smallest element of A that we have yet to include in our list will always be $A[i]$, and similarly the smallest element of B we have yet to include will be $B[j]$.

Thus, each iteration of the while loop takes the smallest element we have yet to add to Result and copies it to Result. Once one of A or B runs out of elements to copy (i.e. i reaches $m + 1$ or j reaches $n + 1$), we take all the remaining elements from the other list. Thus, since at every step we are adding the smallest element we have yet to add, at the end of the while loop Result is a sorted list containing all the elements of A and B combined.

Each iteration of the while loop runs in constant time, since the values we are copying and incrementing can be expressed in a constant number of words, so since the while loop runs $m + n$ times, the total runtime is $\Theta(m + n)$. $\square$

Now we can define MERGESORT, which uses MERGE.

```

MERGESORT( $A[1, \dots, n]$ )
  If  $n < 10$ :
    INSERTIONSORT( $A$ )
    Return
   $m \leftarrow \lfloor n/2 \rfloor$ 
  MERGESORT( $A[1 : m]$ )
  MERGESORT( $A[m + 1 : n]$ )
   $A \leftarrow \text{MERGE}((A[1 : m]), A[m + 1 : n])$ 

```

Theorem 44. For every array A of integers, after calling $\text{MERGESORT}(A)$, A is sorted in increasing order.

Proof. We prove this by induction on n .

- Base case: If $n < 10$, then MERGESORT simply calls INSERTIONSORT. By Theorem 16, INSERTIONSORT correctly sorts A .
- Induction Hypothesis: Suppose that for every array B of length $k < n$, MERGESORT(B) correctly sorts B .

- Induction Step: Suppose $n \geq 10$. Then $A[1 : m]$ and $A[m + 1 : n]$ are both arrays of length less than n , so by our IH, MERGESORT correctly sorts them. By Theorem 43 MERGE correctly combines the sorted subarrays $A[1 : m]$ and $A[m + 1 : n]$ into a sorted version of A .

□

Analyzing the runtime of MERGESORT is more complicated. The call to INSERTIONSORT takes constant time (since we only call it on an array with less than 10 elements), as does computing m (since n is a value which can fit in a single integer variable) and by Theorem 43 the call to MERGE takes $\Theta(n)$, so the total time for everything outside of the recursive calls is $\Theta(n)$. On top of this, however, the algorithm makes 2 recursive calls. We can express the runtime of this algorithm using a *recurrence*:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n).$$

Implicitly we also have a base case $T(n) = \Theta(1)$ if $n < 10$, but since the base case of almost any recurrence describing an algorithm's runtime will be “constant time when the input size is constant” we generally omit this. In the next section we'll see how to use this recurrence to derive a non-recursive expression for the runtime of MERGESORT.

8.4 Analyzing the Runtime of Divide and Conquer Algorithms

Using induction to prove a recurrence

Theorem 45. *The runtime of MERGESORT is $O(n \log n)$*

Proof. We noted in the previous part that MERGESORT satisfies the recurrence $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n)$, so we need to justify that this recurrence satisfies $T(n) = O(n \log n)$. Let $f(n)$ be the exact function hidden by the $\Theta(n)$ term in the recurrence for T . By the definition of big- Θ there exist constants c and n_0 such that $\forall n \geq n_0, f(n) \leq cn$. Let $d := \max_{1 \leq n < n_0} f(n)$. Then $\forall n \in \mathbb{Z}^+, f(n) \leq cn + d$, and thus $T(n) \leq T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + cn + d$. Let $b := \max_{0 \leq n < 10} T(n)$ be the maximum number of steps taken by MERGESORT in the base case of the recurrence.

We will prove by induction on i that for every positive integer i , $T(2^i) \leq (b + c + d)2^i \log_2 2^i$. Note that when for any $i > 0$, $\lfloor 2^i/2 \rfloor = \lceil 2^i/2 \rceil = 2^{i-1}$.

- Base Case: Suppose $1 \leq i \leq 3$. Then $2^i < 10$, so $T(2^i) \leq b \leq (b + c + d)2^i(\log_2 2^i)$.
- Induction Hypothesis: Suppose that for every $0 \leq j < i$, $T(2^j) \leq (b + c + d)2^j \log_2 2^j$.
- Induction Step: Suppose $i > 3$. Then we have that

$$\begin{aligned} T(2^i) &\leq T(\lfloor 2^i/2 \rfloor) + T(\lceil 2^i/2 \rceil) + c2^i + d \\ &= 2T(2^{i-1}) + c2^i + d \\ &\leq 2(b + c + d)2^{i-1} \log_2(2^{i-1}) + c2^i + d && \text{(by our IH)} \\ &= (b + c + d)2^i(\log_2(2^i) - 1) + c2^i + d \\ &\leq (b + c + d)2^i(\log_2(2^i) - 1) + (b + c + d)2^i \\ &= (b + c + d)2^i \log_2 2^i \end{aligned}$$

Thus, we have upper bounded $T(n)$ with a constant multiple of $n \log n$ when n is a power of 2. To extend this for n not a power of 2, we note that $T(n)$ is a non-decreasing function and we can upper bound any positive integer n with a power of 2 (specifically $2^{\lceil \log_2 n \rceil}$ which is less than $2n$, so

$$T(n) \leq T(2^{\lceil \log_2 n \rceil}) \leq (b + c + d)2^{\lceil \log_2 n \rceil} \log_2 2^{\lceil \log_2 n \rceil} \leq (b + c + d)(2n) \log_2(2n) \leq 4(b + c + d)n \log_2 n.$$

Thus, we can upper bound $T(n)$ with $4(b + c + d)n \log_2 n$ for every integer $n \geq 2$, so $T(n) \in O(n \log n)$. □

Some notes on this proof:

- We could prove that $T(n) = \Omega(n \log n)$ (and thus $T(n) = \Theta(n \log n)$) using a very similar technique. Instead of upper bounding $\Theta(n)$ with $cn + d$, we lower bound it with $cn - d$, for different constants c and d . Instead of upper bounding $T(n)$ with $T(2^{\lceil \log_2 n \rceil})$ we lower bound it with $T(2^{\lfloor \log_2 n \rfloor})$.
- The technique of upper bounding $T(n)$ with a sufficiently “nice” number (a power of 2 in this case) to avoid dealing with floors and ceilings without affecting the asymptotic behavior of the recurrence applies broadly when solving similar recurrences. Therefore, in the future if we have a recurrence like $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n)$ we will simply write it as $T(n) = 2T(n/2) + \Theta(n)$ since avoiding floors and ceilings is cleaner and does not affect the asymptotic runtime.
- This type of induction proof worked because we knew that the correct runtime was $\Theta(n \log n)$, but it doesn’t help us figure out that the answer to the recurrence is $n \log n$. There are other techniques for doing that, like recursion trees, which you may have seen in previous courses. For the sake of time, we will not cover those techniques in this course. Instead, we will see a theorem in the next section which allows us to quickly obtain the correct answer for most recurrences which arise from divide and conquer algorithms.

8.5 The Master Theorem

Theorem 46. Let $a, b, e \in \mathbb{Z}^+$, $c, d \in \mathbb{R}_{\geq 0}$ be constants, and let T be a recurrence of the form

$$\begin{aligned} T(n) &= aT(n/b) + \Theta(n^c \log^d n) && \text{for } n \geq e \\ T(n) &= \Theta(1) && \text{for } 0 \leq n < e \end{aligned}$$

Then,

1. If $c < \log_b a$, $T(n) = \Theta(n^{\log_b a})$
2. If $c = \log_b a$, $T(n) = \Theta(n^c \log^{d+1} n)$
3. If $c > \log_b a$, $T(n) = \Theta(n^c \log^d n)$

The proof of the master theorem is fairly involved and relies on simpler techniques for solving recurrences that we have intentionally avoided discussing in this course. If you are interested in details on how the master theorem is derived, Section 4.6 of CLRS is a good starting point.

The high level intuition of the theorem is that to solve a recurrence we need to compare the work done in the initial call (ignoring the recursive calls) with the work done across all of the base cases (a recurrence like mergesort’s which repeatedly splits in two will have many different recursive calls reach the base case).

- Higher values of a correspond to more work done in the base cases, since a is the number of times that our function calls itself recursively, and more recursive calls corresponds to more calls to the base case.
- Higher values of b correspond to more work done in the initial call, since b determines how much smaller the subproblems are compared to our original problem, i.e. how fast we reach the base case.
- Higher values of c correspond to more work done in the initial call, since the n^c term is most impactful in the initial call and has no influence over the work at the base case.

There are three possible cases that the recurrence can fall into based on the relative values of a , b , and c .

- The first case is where the influence of a wins, and the time for the recurrence is dominated by the base cases. In this case, c and d don't show up in the runtime, the asymptotic time is just the number of base case calls
- The second case is where the competing influences of a vs. b and c are exactly equal, so the time spent on the base cases ($n^{\log_b a}$) is about the same as the time spent in the initial call ($n^c \log^d n$), and in fact this is the same amount of time spent at each of the $\log n$ levels of recursion. Thus, the total runtime is equal to the work of the initial call times $\log n$, $\Theta(n^c \log^{d+1} n)$. Note that in this case this is the same as $\Theta(n^{\log_b a} \log^{d+1} n)$.
- In the third case, the influence of b and c wins. The size of the subproblems shrinks fast enough, that the work of the initial call dominates the runtime, and so the asymptotic runtime is just the non-recursive work of the initial call.

In the recurrence for mergesort, $a = 2$, $b = 2$, $c = 1$, and $d = 0$ (and $e = 10$, but this doesn't actually impact the asymptotic runtime). Thus, plugging it into the theorem, we see that $c = \log_b a$, so mergesort falls into case 2 (the balanced case), and the asymptotic runtime is $\Theta(n \log n)$, as we proved in Theorem 45. The recurrence for binary search would be $T(n) = T(n/2) + \Theta(1)$ ($a = 1, b = 2, c = 0, d = 0$, which also falls into case 2, yielding the runtime of $\Theta(\log n)$ which we derived in Theorem 41.

Not all recurrences can be solved using the master theorem, but it works for most recurrences arising from natural divide and conquer algorithms, including all of the recurrences you will be expected to derive an asymptotic bound for in this course.

8.6 Comprehension Questions

1. What is the recurrence for Binary Search?
2. What is the runtime of FASTSQUAREROOT? (Give a tight Big-O bound)
3. What is the recurrence for Mergesort?
4. Let $T(n) = 1$ for $n < 10$ and $T(n) = 3T(n/3) + \log_3 n$ for $n \geq 10$. Give a closed-form big- Θ bound for $T(n)$.
5. Let $T(n) = 1$ for $n < 3$ and $T(n) = 4T(n/2) + n \log_2 n$ for $n \geq 3$. Give a closed-form big- Θ bound for $T(n)$.

8.7 Exercises

1. Consider the following algorithm for finding the minimum element of an array:

```

FINDMIN( $A[1, \dots, n]$ )
  If  $n = 1$ :
    Return  $A[1]$ 
   $B, C \leftarrow$  empty integer arrays
  For  $i$  from 1 to  $n$ :
    If  $i \bmod 3 \neq 1$ :
      Add  $A[i]$  to the end of  $B$ 
    If  $i \bmod 3 \neq 2$ :
      Add  $A[i]$  to the end of  $C$ 
  Return  $\min(\text{FINDMIN}(B), \text{FINDMIN}(C))$ 

```

- (a) Give a recurrence for the runtime of this algorithm and justify why the recurrence represents the algorithm's runtime (assume that adding an element to the end of an array is implemented efficiently and does not require resizing the array for every single addition).
- (b) Solve your recurrence to give a big- Θ expression for the runtime of FINDMIN.

2. For $2 \leq i \leq n-1$, we say that an element $A[i]$ of an array $A[1..n]$ is a *local minimum* if $A[i-1] \geq A[i] \leq A[i+1]$. For example, in the array

$$A = [10, 6, 1, 5, 6, 8, 12, 12, 15, 14, 8, 7, 9, 11, 12],$$

indices 3, 8, and 12 correspond to local minimums

Give an algorithm which takes as input an array $A[1..n]$, for some $n \geq 3$, where $A[1] \geq A[2]$ and $A[n-1] \leq A[n]$, and finds the index of a local minimum in this array. Your algorithm should be as efficient as possible (asymptotically). [Hint: Why do the given conditions guarantee that a local minimum exists?]

3. Give a recurrence for the runtime of the following algorithm, then solve it to obtain a big- Θ expression:

```

TRIMERGESORT( $A[1..n]$ )
  If  $n = 1$ :
    Return
   $B_1 \leftarrow A[1.. \lfloor n/3 \rfloor]$ 
   $B_2 \leftarrow A[\lfloor n/3 \rfloor + 1.. \lfloor 2n/3 \rfloor]$ 
   $B_3 \leftarrow A[\lfloor 2n/3 \rfloor + 1.. n]$ 
  TRIMERGESORT( $B_1$ )
  TRIMERGESORT( $B_2$ )
  TRIMERGESORT( $B_3$ )
   $i \leftarrow n$ 
  While  $i > 0$ :
     $j \leftarrow$  the index of whichever of  $B_1, B_2, B_3$  has the largest last element
     $A[i] \leftarrow$  the last element of  $B_j$ 
    remove the last element of  $B_j$ 
     $i \leftarrow i - 1$ 

```

4. Let $T(n) = 3T(n-1) + \Theta(n)$. Prove that $T(n) = O(3^n)$.
5. Let $T(n) = 2T(\sqrt{n}) + \Theta(1)$. Prove that $T(n) = O(\log n)$
6. Suppose you are given two sorted arrays of integers $A[1..n]$ and $B[1..n]$ such that all $2n$ elements of these arrays are distinct. Describe a fast algorithm (better than linear time) to find the median (meaning the n th smallest element) of the union $A \cup B$. For example, given the input

$$A[1..8] = [0, 1, 6, 9, 12, 13, 18, 20] \quad B[1..8] = [2, 4, 5, 8, 17, 19, 21, 23]$$

your algorithm should return the integer 9. (Hint: What can you learn by comparing one element of A with one element of B ?)

7. For an array $A = [a_1, \dots, a_n]$, we define the k -step rotation of A as the array $[a_{k+1}, a_{k+2}, \dots, a_{n-1}, a_n, a_1, a_2, \dots, a_k]$ formed by taking the first k elements of A and moving them to the end of the array. Give an algorithm which takes as input an array of n distinct integers which is a k -step rotation of a *sorted* array (for an unknown value of k) and outputs the number of steps that the sorted array was rotated.

For example, given the array

$$A = [18, 20, 37, 41, 82, 95, 105, 3, 7, 10, 11],$$

your algorithm should output 4, because this array is formed by rotating a sorted array 4 steps. Your algorithm should run in time $O(\log n)$. Briefly justify the correctness and runtime of your algorithm.

8. We say that an array $A[1, \dots, n]$ of integers is a **mountain** if there exists some index i such that for every $1 \leq j < i$ we have $A[j] < A[j + 1]$, and for every $i \leq j < n$ we have $A[j] > A[j + 1]$. That is, a mountain consists of a strictly increasing sequence followed by a strictly decreasing sequence. For example,

$$[1, 3, 10, 7, 5, 4, -2, -7, -15]$$

is a mountain, but

$$[5, 4, -3, 8, 10]$$

is not. Give and analyze an algorithm which takes as input an array A which is a mountain, as well as an integer x and returns either

- An index j such that $A[j] = x$, or
- -1 if x is not in A .

You may assume that all of the elements of A are distinct. Your algorithm must run in $O(\log n)$ time.

9. Let $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ be a strictly increasing function (meaning that $\forall a, b \in \mathbb{Z}^+$ with $a < b$, $f(a) < f(b)$). Suppose that you can compute $f(x)$ in time $O(x)$ for any positive integer x (note that this implies that $f(x)$ has at most $O(x)$ digits). You are given a number $y \in \mathbb{Z}^+$, and you are told that $\exists z \in \mathbb{Z}^+$ such that $f(z) = y$. Give an $O(z \log z)$ time algorithm to find this z .